
title: “Parcial 2021 (03/12/2021) — Resuelto con metodología detallada” subtitle: “Arquitectura de Computadoras FING UdelaR — aplicando la Guía de Notación y Redacción Exacta”

Parcial 2021 (03/12/2021) — Resuelto con metodología detallada

Formato de cada pregunta: Lectura del problema → Qué necesito para resolverlo → Qué cálculo se hace → Qué consideraciones se usan. Si la pregunta es teórica, se detallan las definiciones en lugar de un cálculo.

Práctico(s) relacionado(s) señalado(s) al final de cada pregunta.

Pregunta 1 — Unidad de Control (teórica)

Enunciado: *"Explique qué tipo de circuito es la Unidad de Control de una CPU y qué acciones realiza. Describa los estilos de diseño de Unidad de Control vistos en el curso."*

LECTURA DEL PROBLEMA

Pide tres cosas encadenadas: (1) clasificar la UC como tipo de circuito, (2) decir qué hace (su función), (3) describir las dos alternativas de diseño que vio el curso.

QUÉ NECESITO PARA RESOLVERLO

No hace falta ningún dato del enunciado — es una pregunta puramente conceptual. Necesito tener claras las **definiciones exactas** de:

- Circuito combinacional vs. circuito secuencial.
- Ciclo de instrucción.
- Control cableado vs. microprogramación.

DEFINICIONES (POR SER TEÓRICA)

Unidad de Control (UC): es una **máquina secuencial** (no combinacional, porque su salida en cada instante depende no solo de las entradas actuales sino del estado interno — en qué paso del ciclo de instrucción se encuentra).

Acción que realiza: ejecutar el **ciclo de instrucción** — el conjunto ordenado y secuencial de acciones necesario para completar una instrucción: aplicar la operación indicada sobre los operandos correspondientes y almacenar el resultado en el lugar indicado.

Estilo 1 — Control cableado: la UC se diseña como un circuito secuencial “a mano”, típicamente con la **metodología de Máquinas de Estado** (diagrama de estados → tabla de excitación → Karnaugh → circuito con flip-flops, la misma metodología general de circuitos secuenciales).

Estilo 2 — Microprogramación: la UC se construye en base a una **máquina secuencial más simple** que ejecuta **microprogramas**. Cada microprograma contiene la secuencia de valores que deben tomar las **señales internas de control** de la CPU para lograr ejecutar cada instrucción del set de instrucciones.

CONSIDERACIONES

- No hace falta profundizar en microarquitectura específica (registro de microinstrucción, dirección de próxima microinstrucción, etc.) salvo que el enunciado lo pida explícitamente — acá alcanza con la definición conceptual de ambos estilos.
- Redacción esperada: nombrar primero la clasificación (máquina secuencial), después la función (ciclo de instrucción), y recién ahí desarrollar los dos estilos **en el mismo orden en que se los nombra**.

Práctico relacionado: conceptos de M5 (Von Neumann / CPU) — no hay práctico dedicado, pero conecta con `Apuntes.pdf`, sección “Organización de la CPU”.

Pregunta 2 — Localidad espacial en recorrido de matriz

Enunciado: dos códigos que recorren `short arr[1024][100]` en distinto orden de índices, caché de 1KB. Preguntan cuál aprovecha mejor la localidad espacial y por qué.

LECTURA DEL PROBLEMA

Hay que comparar dos formas de recorrer una matriz (por columnas vs. por filas) contra cómo se almacena realmente la matriz en memoria, y explicar el efecto sobre la caché.

QUÉ NECESITO PARA RESOLVERLO

- **Dato de la estructura:** `arr[1024][100]`, tipo `short` (2 bytes).
- **Dato de la caché:** 1 KB de tamaño.
- **Conocimiento previo necesario:** cómo se almacena una matriz en C en memoria (por filas, en orden contiguo — esto no está en el enunciado, es conocimiento de base que hay que aportar), y la definición del **principio de localidad espacial**.

QUÉ CÁLCULO SE HACE

Tamaño total de la estructura:

$$1024 \times 100 \times 2 \text{ bytes} = 204800 \text{ bytes} = \mathbf{200 \text{ KBytes}}$$

(mostrando la cuenta entre paréntesis junto al resultado, como es la convención del curso: "200 KBytes (1024 x 100 x 2 bytes)").

Como 200 KB $\gg$ 1 KB (tamaño de la caché), la estructura **no entra completa** en la caché — por lo tanto el orden de recorrido sí importa.

Análisis de cada código:

- **Código 2** (recorre `arr[i][j]` con `i` externo 0..1023, `j` interno 0..99): accede a las posiciones de memoria **en el mismo orden en que están almacenadas** (fila por fila, ya que C guarda las matrices por filas). Cada miss trae un bloque de memoria contiguo que después se reutiliza en los accesos siguientes de esa misma fila → **aprovecha bien** la localidad espacial.
- **Código 1** (recorre `arr[j][i]` con `i` externo 0..99, `j` interno 0..1023): en cada paso del bucle interno salta de fila en fila (columna fija), accediendo a posiciones de memoria muy separadas entre sí. Cada acceso cae probablemente en un bloque distinto → **no aprovecha** la localidad espacial.

Demostración alternativa cuantitativa (frase fija del curso: "Una forma alternativa de razonar este resultado es..."): suponiendo una línea de caché de 16 bytes = 8 elementos `short`:

- Código 2: el primer acceso a `arr[0][0]` da **miss** y trae a caché `arr[0][1] ... arr[0][7]` → los próximos 7 accesos son **hit**. El patrón se repite igual en cada fila: 1 miss cada 8 accesos.
- Código 1: con correspondencia totalmente asociativa (simplificación), cada fila nueva (`arr[0][0]`, `arr[1][0]`, `arr[2][0]` ...) da miss y trae un bloque nuevo. Al llegar a `arr[64][0]` la caché (64 líneas de 16 bytes) ya está llena, y el próximo acceso reemplaza el bloque de `arr[0][0..7]` — que se va a necesitar de nuevo apenas termine la primera vuelta del bucle interno. El resultado es que **prácticamente todos los accesos dan miss**.

CONSIDERACIONES

- El resultado no depende del tamaño exacto de línea de caché elegido para el ejemplo (16 bytes es arbitrario, ilustrativo).
- Se asume correspondencia totalmente asociativa **solo para simplificar el razonamiento cuantitativo** — la conclusión cualitativa (Código 2 gana) no depende de eso.

- El dato clave que hace que la pregunta tenga respuesta no trivial es que la estructura **no entra** en la caché (200KB vs 1KB) — si entrara completa, el orden de recorrido no importaría.

Práctico relacionado: Práctico 11, Ejercicios 1-2 (localidad espacial y temporal en bucles anidados).

Pregunta 3 — Construcción de ROM 1Kx12 desde ROMs 2Kx6

Enunciado: "¿Cuántas ROMs de 2Kx6 se precisan para construir una ROM de 1Kx12? Dibuje el circuito correspondiente."

LECTURA DEL PROBLEMA

Piden combinar varias ROMs chicas para armar una ROM más grande de organización específica, y dibujar la conexión.

QUÉ NECESITO PARA RESOLVERLO

- Organización de la ROM disponible: **2Kx6** (2048 palabras, 6 bits cada una).
- Organización de la ROM objetivo: **1Kx12** (1024 palabras, 12 bits cada una).
- Metodología: traducir cada organización a cantidad de líneas de dirección (entradas) y de datos (salidas), y comparar.

QUÉ CÁLCULO SE HACE

Aplicando la plantilla fija de conteo de bits del curso:

Tenemos ROMs de **2Kx6**, por lo que tenemos **11 entradas de dirección** ($2K=2^{11}$) y **6 salidas de datos**. Queremos una ROM de **1Kx12**, por lo que queremos **10 entradas de dirección** ($1K=2^{10}$) y **12 salidas de datos**.

Comparando: se necesitan **menos** líneas de dirección (10 en vez de 11) pero **más** líneas de datos (12 en vez de 6) → el objetivo es **crecer en ancho de palabra**, no en cantidad de palabras. Esto se logra poniendo ROMs **en paralelo** (mismo bus de direcciones a todas, salidas concatenadas), no con decodificación externa.

Cantidad de ROMs necesarias: cada ROM de 2Kx6 aporta 6 de los 12 bits de salida requeridos → se necesitan **2 ROMs de 2Kx6**, una para los bits `D[5..0]` (parte baja de la palabra) y otra para `D[11..6]` (parte alta).

CONSIDERACIONES

- Cada ROM 2Kx6 tiene **11** entradas de dirección ($A[10..0]$), pero la ROM objetivo solo necesita **10** ($A[9..0]$) — sobra una línea de dirección por ROM.
- Esa línea sobrante (A_{10} , la más significativa) se conecta a una **constante conocida** — en este caso a **tierra (GND = 0)** — porque solo se va a usar la mitad inferior de la capacidad de cada ROM (2K de capacidad disponible, pero solo 1K de contenido útil).
- Las señales CS (Chip Select) y OE (Output Enable) de ambas ROMs se conectan igual (juntas, a las mismas señales de control de la ROM compuesta), porque **ambas mitades de la palabra se leen simultáneamente**, no una u otra.

Práctico relacionado: Práctico 4 (Memorias ROM), Ejercicio 1 (construir ROM 32K×8 y 16K×16 desde módulos 8K×8 — mismo tipo de razonamiento).

Pregunta 4 — V/F sobre arquitectura Von Neumann (teórica)

Enunciado: dos CPUs distintas (CPU1, CPU2) implementan la **misma caracterización** de una arquitectura Von Neumann. Se pide indicar V/F y justificar sobre: (1) si el mismo binario compilado para CPU1 corre en CPU2, (2) si ambas CPUs tardan la misma cantidad de ciclos de reloj ejecutando el mismo programa.

LECTURA DEL PROBLEMA

Hay que distinguir entre **qué hace** una arquitectura (la caracterización, el contrato que ve el software) y **cómo lo hace** cada implementación concreta (la organización interna, que puede variar).

QUÉ NECESITO PARA RESOLVERLO

Necesito la lista exacta de elementos que componen la “caracterización” de una arquitectura Von Neumann, vista en el curso, y la distinción conceptual arquitectura-vs-implementación.

DEFINICIONES (POR SER TEÓRICA)

Caracterización de una arquitectura Von Neumann: el conjunto de elementos que la definen desde el punto de vista del software: **set de instrucciones, formato de instrucción, set de registros, modos de direccionamiento, manejo de entrada/salida e interrupciones**. Es, en la literatura, “la interfaz entre el hardware y el software” — la información necesaria para escribir programas en lenguaje de máquina.

Implicancia de “misma caracterización”: que ambas CPUs pueden ejecutar todos los programas que cumplan esa especificación, **sin importar en cuál se compiló el binario** — porque el binario está escrito en términos del set de instrucciones y modos de direccionamiento, que son idénticos en ambas.

ÍTEM 1 — VERDADERO

El mismo binario compilado para CPU1 también corre en CPU2, porque la caracterización (lo que el binario “ve”) es la misma en ambas. El binario no depende de detalles de implementación, solo de la arquitectura.

ÍTEM 2 — FALSO

Que la arquitectura sea la misma **no dice nada sobre cómo está implementada** (organización interna). Ejemplos concretos de diferencias de organización que sí afectan la cantidad de ciclos de reloj, aunque la arquitectura sea idéntica:

- Una CPU puede tener *pipeline* y la otra no.
- Pueden implementar ciclos de instrucción distintos (más o menos etapas).
- La Unidad de Control de una puede estar diseñada por **control cableado** y la de la otra por **microprograma** (ver Pregunta 1).

CONSIDERACIONES

- La palabra clave que decide la respuesta del segundo ítem es la distinción **qué vs. cómo**: la arquitectura especifica **qué** hace cada instrucción, no **cómo** ni en cuánto tiempo la ejecuta el hardware.
- Patrón de redacción esperado para V/F: primero la palabra sola (“Verdadero.” / “Falso.”), después el desarrollo.

Práctico relacionado: conceptos de M5, en particular la distinción arquitectura/organización de [Apuntes.pdf](#) (“Máquina Lógica General” y “Arquitectura Von Neumann”).

Pregunta 5 — Identificar función de un MUX 4x1 e implementarla

Enunciado: identificar qué función booleana implementa un mux 4x1 con las entradas de datos conectadas fijas a / de una manera dada, y luego implementarla en dos niveles con AND/OR/NOT.

LECTURA DEL PROBLEMA

El mux tiene sus 4 entradas de datos $D0..D3$ cableadas a valores fijos (no a variables), y las líneas de selección son las variables a, b . Hay que leer del diagrama qué valor fijo tiene cada Di , armar la tabla de verdad resultante, identificar la función, y después sintetizarla con compuertas.

QUÉ NECESITO PARA RESOLVERLO

- Lectura del diagrama: $D0=GND(0)$, $D1=GND(0)$, $D2=Vcc(1)$, $D3=Vcc(1)$ (según la descripción del circuito dado).
- Conocimiento de cómo funciona un mux 4x1: la salida y toma el valor de la entrada Di seleccionada por las líneas de control (a, b forman el índice i en binario).
- Metodología de identificación de funciones booleanas a partir de tabla de verdad (suma de productos canónicos).

QUÉ CÁLCULO SE HACE

Armar la tabla de verdad recorriendo las 4 combinaciones de a, b y leyendo qué Di selecciona cada una, y qué valor tiene ese Di :

a	b	función(a,b)
0	0	1
0	1	0
1	0	0
1	1	1

Identificación: esta tabla es la **negación del XOR**, es decir, **XNOR** — la función que vale 1 exactamente cuando a es igual a b .

Síntesis en dos niveles (suma de productos canónicos): se toman los minitérminos donde la función vale 1 (filas 00 y 11):

$$\text{función}(a,b) = a.b + !a.!b$$

Circuito: dos inversores (para $!a$ y $!b$), dos compuertas AND de 2 entradas (una con a, b , otra con $!a, !b$), y una compuerta OR final que las combina.

CONSIDERACIONES

- La mínima implementación en dos niveles **coincide** con la suma de productos canónicos en este caso particular (no siempre es así — acá se debe a que la función XNOR de 2 variables no se simplifica más).
- Notación booleana obligatoria del curso: AND con punto `.`, OR con `+`, NOT con `!` antes de la variable (nunca `a'`, nunca barra).
- Nombre de función en minúscula: `función(a,b)`, tal como aparece en el enunciado — no renombrar a `F(a,b)`.

Práctico relacionado: Práctico 3 (Álgebra de Boole y circuitos combinatorios), Ejercicio 12 ($f(x) = -x$ con multiplexores) y la sección de simplificación con Karnaugh/suma de productos canónicos.